Softwarica COLLEGE OF IT & E-COMMERCE | in collaboration with Coventry University

Network Sniffer: Demonstrating Credential Exposure in Unencrypted Network Communications

Coursework By Prabhuti Sharma

BSc. (Hons) Ethical Hacking and Cybersecurity

ST4017CMD Introduction to Programming

Student ID: 250244

ABSTRACT

This project aims to demonstrate the vulnerability of network communications done through an unencrypted network. Using a Python based network sniffer, the risks associated with the use of unsecure connections are exposed and documented. The tool captures and analyses data packets within a local network with focus on HTTP traffic transmitted in plaintext. By inspecting packet payloads, the system identifies login credentials demonstrating how easily sensitive information can be exposed and reinforces the critical importance of encryption, secure communication and cybersecurity practices. This report details the problem statement, methodology, tools used, result and mitigations for aforementioned problem i.e. network security risks.

TABLE OF CONTENTS

Abstract ii
1. Introduction 1
1.1 Background 1
1.2 Problem Statement and Scope 2
Problem Statement 2
Scope of the Study 3
1.3 Objectives of the Study 4
2. Literature Review 4
3. Methodology 5
3.1 Algorithm 5
3.2 Tools and Technology Used 7
3.3 Source Code: 8
3.4 Procedure and Implementation 8
4. Result 13
5. Further Refinement 15
6. Conclusion 16
References 17

TABLE OF FIGURES

Figure 1 Flow of Network Connection with Sniffer 2
Figure 2 Risks of Networking Sniffing 3
Figure 3 Specifications of OS Used 7
Figure 4 Imported Libraries 8
Figure 5 Functionality of Libraries in Network Sniffing 8

1. INTRODUCTION

This project was developed to provide a practical demonstration of how insecure network communication can lead to the exposure of sensitive information. The system monitors traffic within a local network connection, captures packets and parses it with the use of in-built Python libraries. To exemplify the use of a sniffer in the cybersecurity domain, a demonstration of credential exposure is provided using an intentionally vulnerable HTTP server operating without encryption. Proper exception handling is included such that errors are anticipated and dealt with.

1.1 BACKGROUND

The Network World headlined the original network sniffer creation in 1986 with "Success with Sniffer; protocol analyzer making waves in industry"[1]. The start of a "network analysis and diagnostic tool" that captured data packets transmitted across a local-area network was a remarkable venture for what they used to call a network management domain. It was expressed that the product was merely a development tool and only focuses on a very specific kind of problem: taking a look at a local-area network and not a wide-area network. Since then, the progress in network analysis has not halted, matter of fact, they have expanded at an accelerative speed with AI-driven network monitoring and automated threat detection being the topic of discourse in the current timeline.

The original source code for the ARCNET sniffer is till this day available on the internet [2] which has been an inspiration for this project. Now, in the current data-driven world that uses network around the clock, network sniffer is a tool that is essential in fields namely cybersecurity, IT operations, compliance auditing. They help detect intrusions, monitor traffic patterns and investigate security patterns. Additionally, with the rise of cloud computing, visibility into network traffic is more important than ever.

The network sniffer monitors and captures the transmitted traffic that is being transversed following a structural flow. To get a general understanding of the communication flow, figure 1 displays a user device connected to a router or switch, which in turn sends requests to a web server and receives responses in the form of data packets. The said packets transverse multiple points before reaching its destination within an infrastructure. Now the sniffer comes into play at these transitional stages, enabling the interception of analysis of packets in transit.

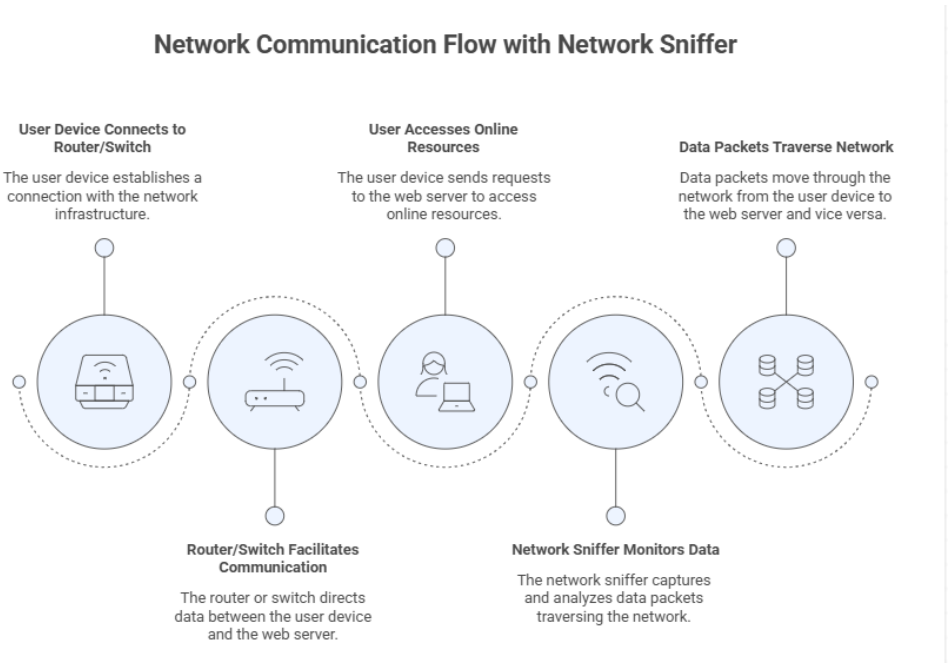


Figure 1 Flow of Network Connection with Sniffer

It is said that network sniffing is a double-edged sword; while it may allow bandwidth auditing and security analysis, it can also be used as a tool to expose plaintext credentials.

1.2 PROBLEM STATEMENT AND SCOPE

PROBLEM STATEMENT

In this interconnected world, plethora of data packets transverses through the realm of network architecture every fraction of a second. For data to transmit seamlessly and securely, great importance is placed on the integrity of the network infrastructure and communication protocols. To succinctly describe what a communication protocol is, it is a standardized set of rules and conventions that dictate how data is exchanged in-between devices or systems in a network. Not all protocols provide built-in security mechanisms and when the data is transmitted using unencrypted protocols such as HTTP, the plaintext information can be intercepted by monitoring tools.

The risks of network sniffing are due to it enabling crimes involving financial data theft, credential theft, espionage and data exfiltration, session hijacking or man-in-the-middle as some may call it and even for further exploits through device mapping. The widespread availability of sniffing tools

combined with insufficient cyber monitoring and flaws in encryption or the lack thereof amplifies these risks, making network sniffing an evolving threat.

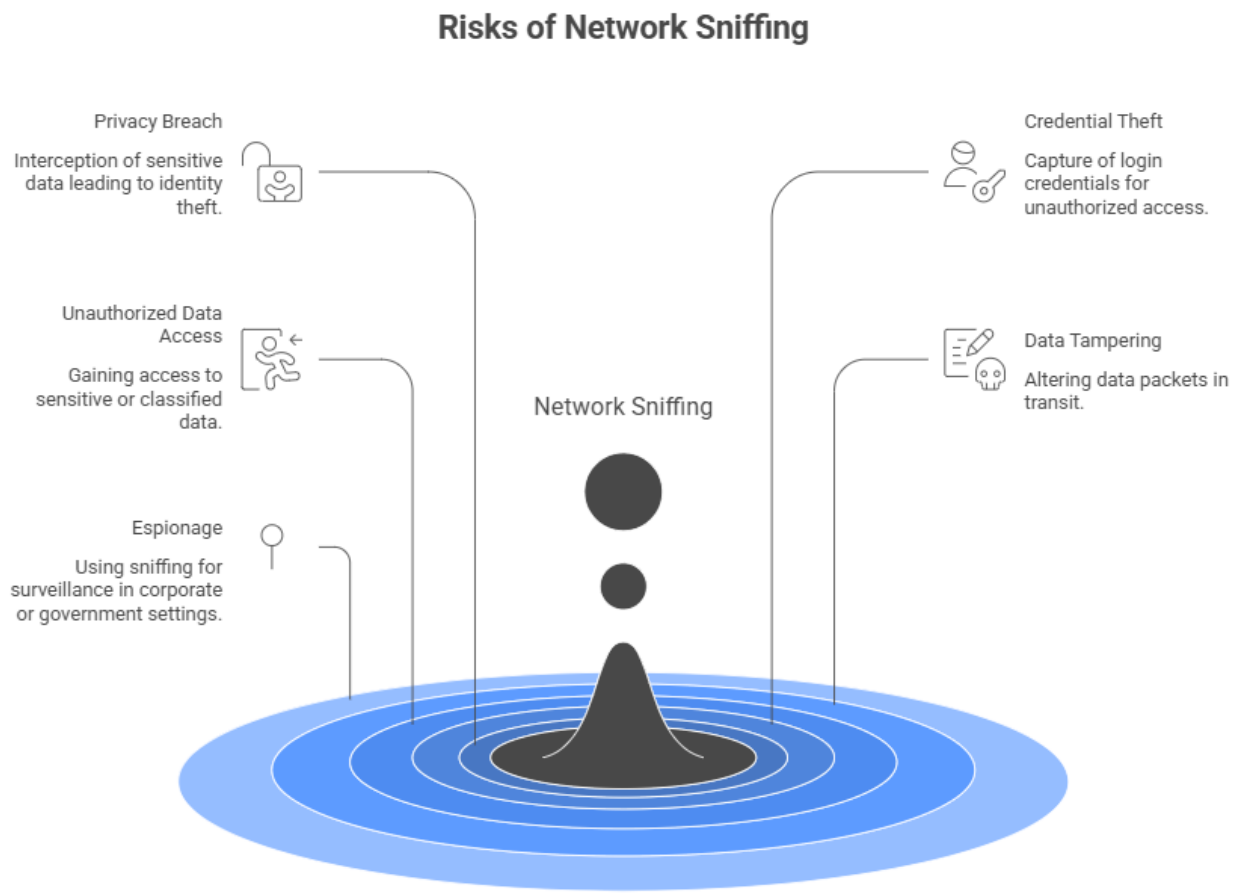


Figure 2 Risks of Networking Sniffing

SCOPE OF THE STUDY

The main scope of this study is to explore network sniffing as both a cybersecurity threat and a tool to analyse network. Understanding the mechanisms of sniffing including packet capture and network mapping and using that insight to expose the vulnerabilities that can be exploited by a cybercriminal. The parameters used in this assessment are the technical aspects of the capture of packets, most common types of attacks such as man in the middle attacks, the involvement of unsecured Wi-Fi networks and the effect of poorly implemented encryption protocols. Through these facets, the project assesses the threats to the people and organizations and points out the possible outcomes such as identity theft, unauthorized access to the systems, and major data breaches. In addition, the paper examines prevention and mitigation measures like adoption of

protection communication protocols (e.g. HTTPS and TLS), network intrusion detection tools, correct setup of network devices and periodic security checks. None of this is meant to imply that the project will fail to offer a systematic insight into how network sniffing works, the dangers that it poses, and the steps that can be taken to mitigate its consequences in the current digital ecosystems.

1.3 OBJECTIVES OF THE STUDY

1. To explore the general concepts of network sniffing and to know how packet capture works in the network infrastructures.
2. To demonstrate real-time packet sniffing using Python and Scapy library in a controlled lab set up.
3. To detect potential plaintext credential exposure in HTTP traffic as a demonstration of insecure data transmission.

2. LITERATURE REVIEW

The fast-faced growth in cyberattacks and the techniques cybercriminals use is evident when terms like “data breach”, “compromised credentials” and “hacked” make the headlines almost daily. This evolution requires advancement in tools for network vulnerability assessments. Packet sniffing was introduced as an idea with the emergence of early computer networks as a way of diagnosing network performance problems. Packet analyzers were initially needed to accomplish legitimate troubleshooting and debugging functions; however, they became needed as tools that would help network administrators to observe the traffic, identify faults and better utilise bandwidth.

As the internet and wireless networks continued to spread, network sniffing tools like Wireshark and tcpdump were easily available. Although these tools can be used in legitimate cybersecurity purposes, studies have indicated that they can also be used to commit malicious intent like intercepting credentials, hijacking a session and stealing financial data [3]. Some of the attack methods such as Man-in-the-Middle (MITM) attacks and ARP spoofing attacks depend on the interception of packets.

As noted in the current studies in cybersecurity, poor encryption, absence of it, and inadequate network surveillance pose a huge risk of being attacked by use of packets. Plaintext credential

exposure has been minimized by the implementation of secure protocols like SSL/TLS and HTTPS, but research also shows that systems with incorrect settings and unsecured open networks are still susceptible.

This coursework seeks to give a practical understanding of the packet sniffing tools, the risks they may present as well as how defensive mechanism can be employed. It enables security professionals to analyse network infrastructures, identify risks and apply the necessary security policies.

3. METHODOLOGY

3.1 ALGORITHM

Step 1: **Start**

i. Initialize by importing required libraries. (scapy, colorama, os, sys, datetime, json)
ii. Check if scapy is installed and terminate program if not, with an error message.
iii. Display banner
iv. Check if administrative privilege is there if not, warn about full functionality.

Step 2: **Initialize data structures**

i. packet_list for processed packet info (linked list).
ii. raw_packets for raw packet objects (for PCAP export).
iii. bandwidth_ip and bandwidth_proto for tracking bandwidth.
iv. stats for protocol packet counts.

Step 3: **Display Main Menu (loop until exit)**

Options: 1: Start Live Capture
2: Show Stats & Bandwidth
3: List Interfaces
4: Export Logs to JSON/PCAP
5: Exit

If user selects:

- Option 1 → Proceed to Step 4
- Option 2 → Proceed to Step 5
- Option 3 → Proceed to Step 6
- Option 4 → Proceed to Step 7
- Option 5 → Proceed to Step 8

Step 4: **Packet Sniffing**

i. Prompt user for network interface (default = all interfaces).

ii. Prompt for number of packets to capture (default = unlimited).

iii. Prompt for optional BPF filter (default = no filter).

iv. Start packet capture in real-time using scapy.sniff().

For each captured packet:

- Extract Ethernet, IP, TCP, UDP, ICMP, ARP, and DNS information.
- Detect TCP flags, ports, and common services.
- Check payload for possible plaintext credentials (HTTP username or password).
- Update protocol statistics and bandwidth per IP and protocol.
- Display packet info
- Store processed packet info in a linked list for JSON export.
- Store raw Scapy packets in a separate list for PCAP export.

v. Stop capture on Ctrl+C.

Step 5: **Stats and Bandwidth**

i. Display total packet count per protocol.

ii. Display bandwidth usage per IP address.

iii. Display bandwidth usage per protocol.

iv. Return to Main Menu.

Step 6: **List Interfaces**

i. Retrieve available network interfaces using scapy.get_if_list().

ii. Display the list of interfaces.

iii. Return to Main Menu.

Step 7: **Export Logs**

i. Prompt user for JSON filename.

ii. Prompt user for PCAP filename.

iii. If no packets were captured, display warning and return to Main Menu.

iv. If JSON filename is provided:

Export processed packet metadata from packet_list into JSON format.

v. If PCAP filename is provided:

Export raw packet objects from raw_packets into PCAP format.

vi. Confirm successful export.

vii. Return to Main Menu.

Step 8: **Exit**

End

3.2 TOOLS AND TECHNOLOGY USED

1. Programming Language:

The programming language used was Python (version Python 3.14.2). Its simplicity and flexibility along with the availability of extensive networking libraries was best-suited for network scanning and packet sniffing.

2. Libraries:

- Scapy (for packet sniffing and protocol analysis)
- datetime (for timestamp logging)
- json (for exporting logs)
- threading (for handling capture processes)
- os (file handling)
- sys (argument handling)
- socket (for IP address)

3. Hardware:

A system with a network interface capable of operating in promiscuous mode for packet capture.

Item	Value
OS Name	Microsoft Windows 11 Home
Version	10.0.26200 Build 26200
Other OS Description	Not Available
OS Manufacturer	Microsoft Corporation
System Name	LAPTOP-I6CC78Q0
System Manufacturer	LENOVO
System Model	83JE
System Type	x64-based PC
System SKU	LENOVO_MT_83JE_BU_idea_FM_LOQ 15IRX10
Processor	13th Gen Intel(R) Core(TM) i5-13450HX, 2400 Mhz, 10 Core(s), 16 Logical Processor(s)
BIOS Version/Date	LENOVO R3CN39WW, 7/24/2025
SMBIOS Version	3.4
Embedded Controller Version	1.39
BIOS Mode	UEFI
BaseBoard Manufacturer	LENOVO

Figure 3 Specifications of OS Used

4. Development Environment:

This code was written, debugged and edited in Visual Studio Code to be a command line interface (CLI) based tool.

3.3 SOURCE CODE:

Github: https://github.com/cyberkitsunex/networkpossum/tree/main

3.4 PROCEDURE AND IMPLEMENTATION

1. Setup and Initialization

Ensure the system has Python 3.x installed. Import the required libraries:

```
import sys
import os
import datetime
import json
import threading
from colorama import init, Fore, Style

# Check if scapy is installed
try:
    from scapy.all import *
    except ImportError:
    print("Error: scapy not found. Install with: pip install scapy")
    sys.exit(1)
```

Figure 4 Imported Libraries

The core component of this project is the Scapy library which enables network scanning, tracerouting, probing and even attacks discovery [4]. In this project, scapy handles packet capturing in real-time, protocol layer parsing and identification, payload analysis and saves the raw packets for further review. As the code relies heavily on scapy, in order to prevent errors in the case of scapy not being available, error-handling is done. Each of the library has its own functionality as addressed by figure 4.

Figure 5 Functionality of Libraries in Network Sniffing

2. Functions

1. print_banner() – showcases the tool name “NetworkPossum: Network Sniffer”. This visually appeals towards the users and clearly pinpoints the tool’s purpose.

```
# Banner Function
def print_banner():
    init(autoreset=True)
    print(Fore.RED + Style.BRIGHT + "="*70)
    print(Fore.RED + Style.BRIGHT + "        ⚠ NETWORKPOSSUM - Network Sniffer Active ⚠")
    print(Fore.RED + Style.BRIGHT + "="*70)
    print(Fore.GREEN + "[+] Sniffing network traffic...")
    print(Fore.GREEN + "[+] Monitoring packets in real-time...")
    print(Fore.YELLOW + "[!] Use responsibly and ethically.")
    print(Fore.RED + "="*70 + "\n")
```

This function utilizes the colorama library for coloring and styling text in the terminal. init(autoreset=True) parameter is called to automatically reset text color and style after each print statement, preventing color bleed in subsequent outputs.

2. check_privileges() – os.name returns "nt" on Windows and "posix" on Linux/macOS. As packet sniffing requires permission, particularly on Windows systems using Npcap, this ensures that the program is executed with the required system privileges. It displays a warning if not.

```
# Check if running as admin (npcap may block raw packet capture, admin access all network interfaces)
def check_privileges():
    if os.name == "nt":
        print("Note: For full packet capture functionality, run as Administrator.")
```

3. Data Structures –

In the program, the two user-defined data structures are applied rather than the Python-built-in list and dict.

3.1 PacketNode –

```
class PacketNode:
    """Node for linked list storing packet info"""
    def __init__(self, info):
        self.info = info
        self.next = None
```

PacketNode is the component of the linked list. The data of each packet (a dict) is stored in self.info and the next node reference (self.next) is stored in self.info. Next when made is the None - that is, its successor is at present unknown.

3.2 PacketLinkedList – This class deals with the dynamic storing of packets. Packets are captured and then new nodes are added to the list, allowing unlimited storage size implications. add_packet() appends a new node to the tail. transverse() yields to return one packet dict at a time without loading all data into memory at once.

```
# For Packet Storage
class PacketLinkedList:
    """Linked list for storing captured packets"""
    def __init__(self):
        self.head = None

    def add_packet(self, info):
        node = PacketNode(info)
        if not self.head:
            self.head = node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = node

    def traverse(self):
        current = self.head
        while current:
            yield current.info
            current = current.next
```

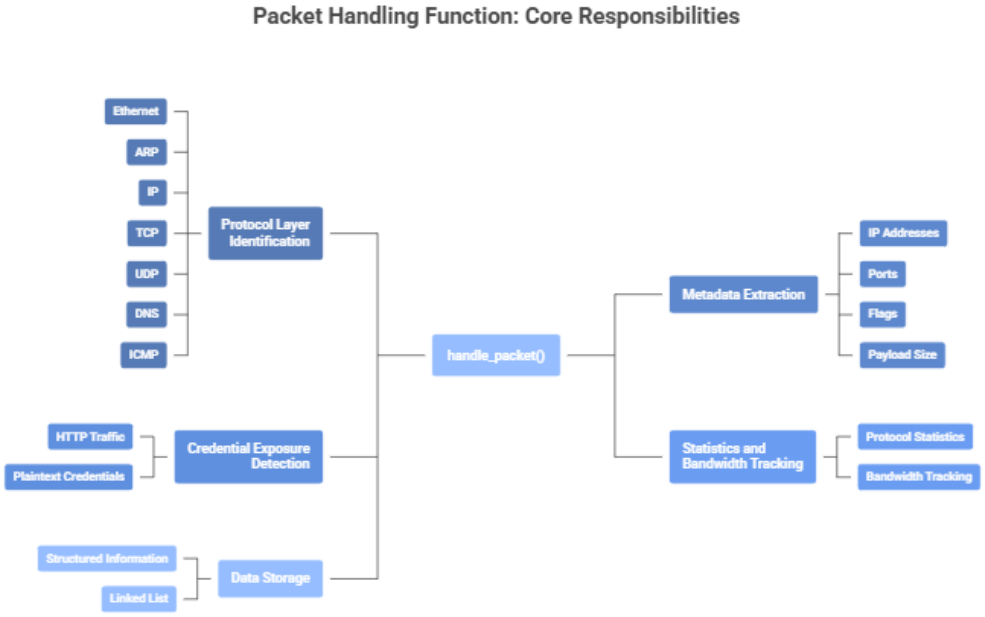
3.3 BandwidthMap – It uses two parallel lists to represent the identifiers (e.g. 192.168.1.1, TCP) and the accumulated number of bytes or number of packets using that key respectively at that index. add is used to add a new key-value pair or to increment an existing key-value pair. It is operated in three modes: bandwidth ip (bytes per source IP), bandwidth proto (bytes per protocol) and stats (number of packets per protocol).

```
# For Bandwidth Tracking
class BandwidthMap:
    def __init__(self):
        self.keys = []
        self.values = []

    def add(self, key, value):
        if key in self.keys:
            idx = self.keys.index(key)
            self.values[idx] += value
        else:
            self.keys.append(key)
            self.values.append(value)

    def get(self, key):
        if key in self.keys:
            return self.values[self.keys.index(key)]
        return 0

    def all_items(self):
        return zip(self.keys, self.values)
```

4. sniff_thread() – This is where the packet capture is initiated which calls sniff() directly, which blocks until count packets are captured or the user presses Ctrl + C.

`iface`	The network interface to listen on. None means all interfaces.
`count`	Number of packets to capture. 0 means capture indefinitely.
`filter`	A BPF (Berkeley Packet Filter) string such as 'tcp port 443' or 'host 8.8.8.8'. Filters are evaluated by the OS kernel before packets reach Python, making them very efficient.
`prn`	A callback function. Scapy calls handle_packet(pkt) for every captured packet.
`store`	False tells Scapy not to keep its own internal copy of packets, saving memory since we handle storage ourselves via packet_list.

3.5 handle_packet() – The captured packets are processed with the function. It is invoked for each of the packets captured by Scapy. Here's where the protocol layers are identified. It also inspects basic security by detecting potential plaintext credential transmissions within HTTP traffic.

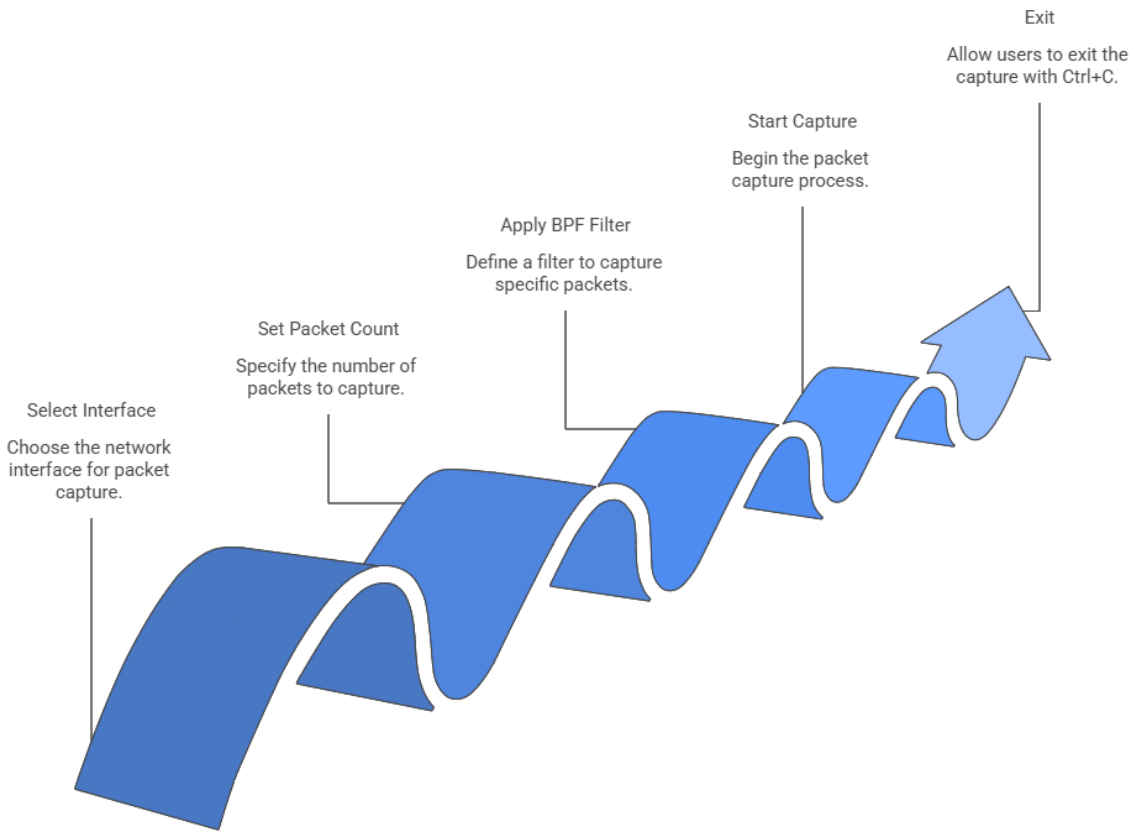


4. print_stats() – This generates a summary including total packet counts per protocol, bandwidth usage per IP address, and bandwidth usage per protocol.

5. list_interfaces() – A scapy function to display all available network interfaces detected by Scapy.

6. main() – Now, this is the entry point that runs an infinite while loop when True presenting the main menu for the toolkit. User's choice is taken with input() and the input dispatches to its correlated function.

Choice 1: Start Live Capture



Choice 2: Show Stats & Bandwidth calls print_stats() to display all accumulated data.

Choice 3: List Interfaces displays the network interfaces captured by Scapy

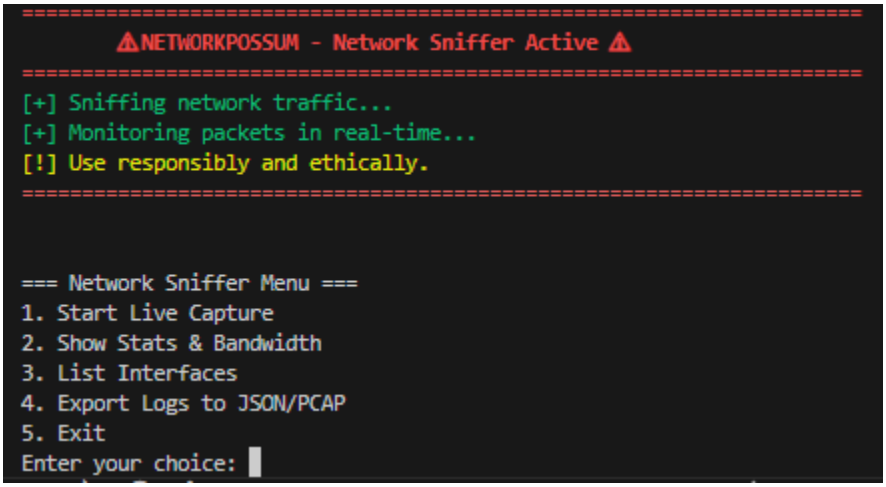
Choice 4: Export Logs is for documentation or further forensic analysis. It exports captured packet data into JSON and PCAP file formats.

Choice 5: Exit terminates the application

4. RESULT

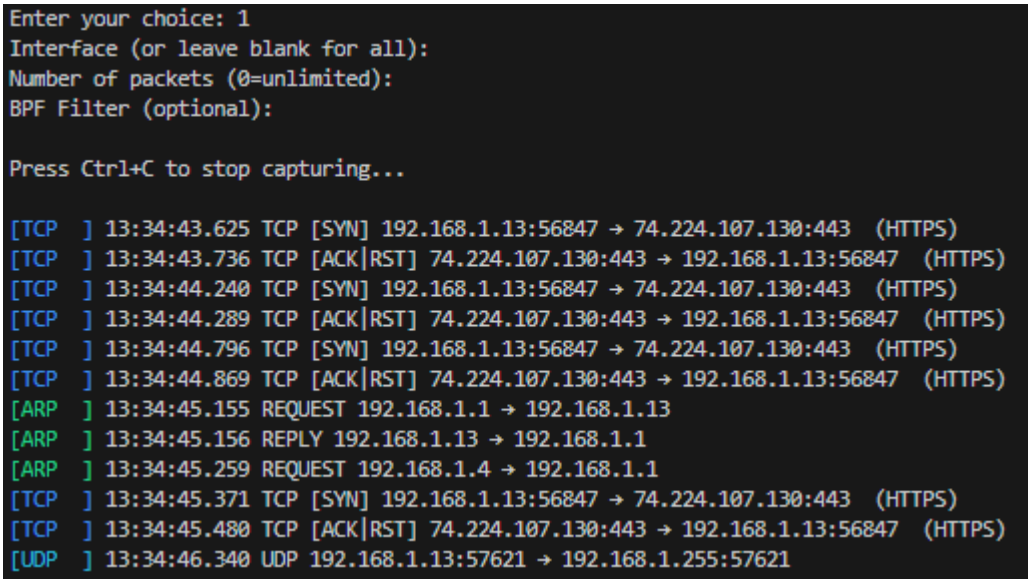
This section showcases the functionality of the tool.

1. Running the tool



We start with the main menu and the banner for our tool. As the structured and color-coded interfaces enhances readability, user can easily navigate through and select one of the five options.

Starting with Option 1, it begins monitoring traffic on the chosen network interface. The system displays protocol-labeled outputs in real-time.

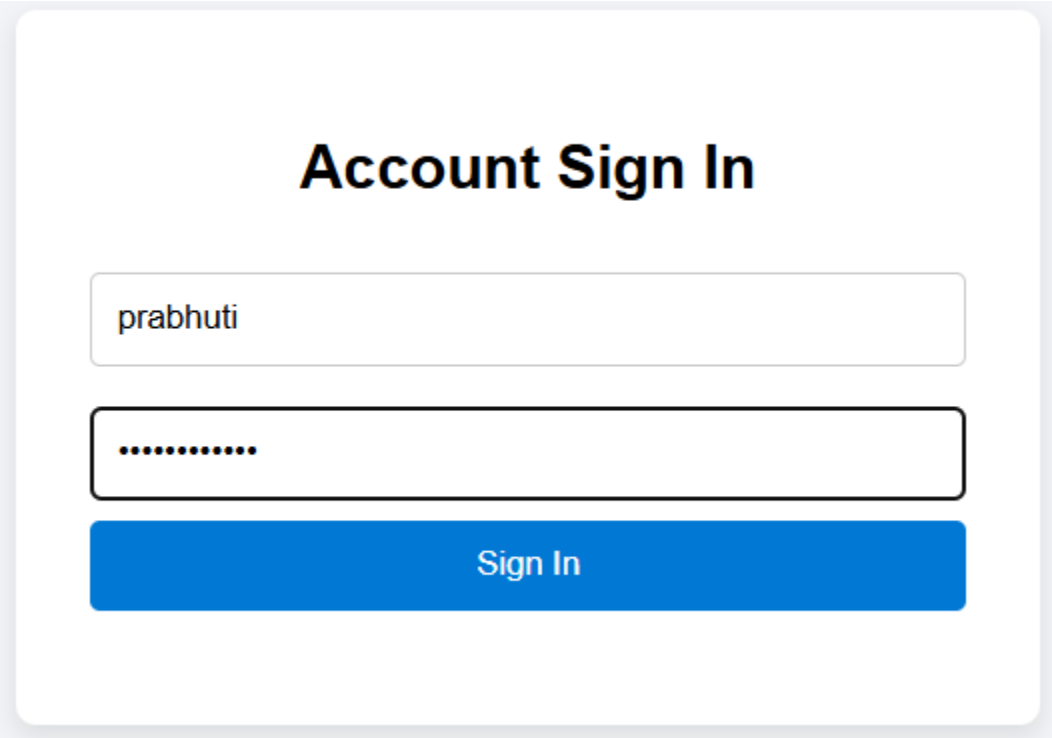


In every output line, there is protocol type, time stamp, source and destination addresses and pertinent port data. In case of TCP packets, there is flags (SYN, ACK, FIN) and payload size. This

output indicates that the tool has been able to successfully read through a variety of protocol layers and operate with dynamically-processed traffic.

Credential Exposure Detection (Security Demonstration)

When performing TCP traffic inspection, especially on the HTTP port 80, the tool determines the content of payloads that may include plaintext credential patterns like a username= or password=. In case of detecting such patterns, warning message is shown that there is a possibility of plaintext credential transmission. This shows that insecure HTTP communication can give out sensitive information, which supports the cybersecurity awareness goal of the project.




```
Username: prabhuti
Password: testpassword
192.168.1.13 - - [01/Mar/2026 17:33:08] "POST / HTTP/1.1" 200 -
192.168.1.13 - - [01/Mar/2026 17:33:08] "GET /favicon.ico HTTP/1.1" 200 -
```

Traffic Analysis and Bandwidth Analysis

Upon the user choosing the statistics option, the system will create a summary report, which will give the total number of packets per protocol and bandwidth used by each IP address and protocol.

This output allows one to analyse predominant patterns of communication in the network. Considering this, more TCP traffic can be taken as an indicator of active browsing sessions and the DNS traffic represents the domain resolution activity. Bandwidth distribution points out the devices that have the highest network traffic.

Enlisting Network Interfaces.

When the interface listing option is selected, all the available network adapters that Scapy identifies are shown. This can be Ethernet, Wi-Fi, virtual adapters or VPN interfaces. This output confirms that there is the option to select the right monitoring source by identifying interfaces capable of capturing dynamically all interfaces.

Exporting Logs (JSON and PCAP Output)

Exporting logs will be possible only after the initial setup finishes.<|human|>Exporting Logs (JSON and PCAP Output) Exporting logs will become viable only after the initial setup is complete. The export option enables capturing packets data to be stored in either structured information or raw PCAP format. The processed metadata of packets is saved in the JSON file and the raw packet data saved in the PCAP file to be analyzed further with other tools like Wireshark. The effective creation of files is a guarantee that the machine used in storage and retrieval of packets is operating correctly and assists in investigating and recording history.

Program Termination

When the exit option is chosen, the application closes.

5. FURTHER REFINEMENT

When the threats keep evolving at an unprecedented pace, it is only natural to find a big room for improvement in the cybersecurity tools we have in our hands. Because this project only delved into the basic understanding of network sniffing, one potential refinement can be performance optimization. To consider in the high-traffic networks will be an a big move as this program only features single handler function as of now.

The credential detection mechanism is limited to identifying plaintext credentials to maintain simplicity for this project. However, future versions can expand to deeper packet inspection techniques (DPI). Also, there are opportunities present in data storage and scalability by

integrating, perhaps a database or a file-based indexing system. This would greatly improve the memory efficiency and the data retrieval speed.

6. CONCLUSION

This project was able to show the vulnerability of unencrypted network communications to eavesdropping sensitive information by use of the packet sniffing method. The study demonstrated the practical example of the interception and analysis of plaintext credentials sent over the HTTP by creating a Python-based network sniffer, based on the Scapy library. The results confirm the significance of safe communication protocols like HTTPS and TLS in ensuring user credentials and confidential information. The test revealed that without encryption, even low-tech surveillance tools can intercept the login details hence pointing out a major security weakness.

Also, the project showed how packet sniffing can be utilized in defense and attack format. It can be used in an unethical and irresponsible manner, negatively affecting the network, but at the same time, it is a potent diagnostic, intrusion detection, and traffic analysis tool in case of ethical and responsible use.

The deployment was only on a controlled lab and simple credential detection systems but it successfully met its intended goals of displaying packet capture, protocol analysis and vulnerability exposures. It may be enhanced in future with more extensive packet inspection, more sophisticated filtering, database-based scalable logging and graphical representation of network statistics. On balance, this coursework offers a conceptual framework on what is network sniffing and why encryption and secure configuration of networks is a critical part of the contemporary digital ecosystem.

REFERENCES

https://books.google.com.np/books?id=aR0EAAAAMBAJ&pg=PA15&redir_esc=y#v=onepage&q&f=false

2 https://github.com/LenShustek/NestarSystems

3 https://rsisinternational.org/journals/ijriss/articles/packet-sniffing-in-the-cyber-threat-landscape-examining-wireshark-capabilities-misuse-and-policy-options-in-the-philippines/

4 https://scapy.readthedocs.io/en/latest/introduction.html